

El efecto Mpemba cuántico en HPC

Modos lentos, evolución temporal, trayectorias y redes tensoriales

Juan | Sebastián

Sistemas Distribuidos / HPC — 2026

Versión completa (≈ 20 min) — implementación serial y paralela (OpenMP · MPI · hilos · CUDA),
validación y escalado

Hoja de ruta

1. De lo clásico (macro) a lo cuántico
2. T1 — Modos lentos del Liouvilliano
3. Evolución temporal (a): RK4 directo
4. Evolución temporal (b): Krylov
5. Muchos cuerpos: trayectorias (MPI) + GPU
6. Muchos cuerpos: redes tensoriales
7. Conclusiones generales
8. Relevancia en el efecto macro/meso

Juan modos lentos · redes tensoriales

Sebastián evolución temporal · trayectorias · GPU

1 · De lo clásico a lo cuántico

Juan

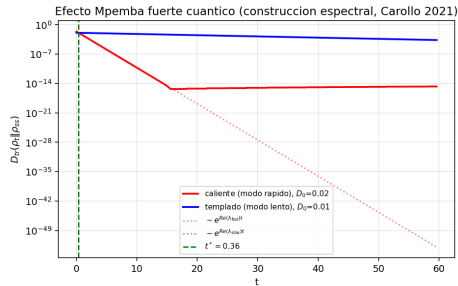
El efecto Mpemba: una anomalía de relajación

Juan

Clásico (macro). A veces el agua *más caliente* se congela *antes* que la templada; se observa también en coloides, granulares e imanes [17, 14, 11, 3, 21].

La idea no es la temperatura sino la **geometría**: si medimos una *distancia al equilibrio*, la curva que parte **más lejos** puede **cruzar** a la cercana y llegar antes.

Cuántico. En sistemas abiertos markovianos reaparece como un cruce de curvas de distancia al estado estacionario [5, 18] — y se puede *predecir* con álgebra lineal.



El cruce de curvas: el corazón del efecto.

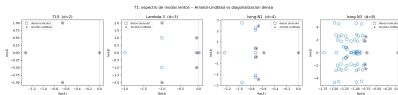
Un sistema cuántico abierto markoviano evoluciona con la **ecuación de Lindblad (GKSL)** [13, 8, 4]:

$$\dot{\rho} = \mathcal{L}[\rho] = \underbrace{-i[H, \rho]}_{\text{coherente}} + \underbrace{\sum_{\mu} \left(L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{ L_{\mu}^{\dagger} L_{\mu}, \rho \} \right)}_{\text{disipador (baño)}}.$$

- Modelo concreto: **cadena de Ising disipativa** $H = -J \sum Z_i Z_{i+1} - h \sum X_i$, con canales locales $\sigma^{\pm}$ (baño térmico).
- $\mathcal{L}$ es **lineal** en $\rho \Rightarrow$ un *superoperador* de dimensión $4^N \times 4^N$. Ahí nace el reto de HPC: 4^N crece muy rápido.
- **Clave:** los canales $\sigma^{\pm}$ locales *no* llevan a $\text{Gibbs}(H)$; el estacionario ρ_{ss} es un estado de *no-equilibrio* (núcleo de $\mathcal{L}$).

Descomposición espectral (autovalores λ_k , $\text{Re } \lambda_k \leq 0$):

$$\rho_t = \rho_{ss} + \sum_{k \geq 2} a_k e^{\lambda_k t} \hat{r}_k, \quad a_k = \text{Tr}(\hat{l}_k^\dagger \rho_0).$$



Espectro λ_k : el modo lento controla el cruce.

- La relajación es una **suma de modos** que decaen; a tiempos largos manda el **modo lento** λ_2 , escala $\tau = 1/|\text{Re } \lambda_2|$.
- **Criterio de Mpemba (fuerte)**: si $a_2 = \text{Tr}(\hat{l}_2^\dagger \rho_0) = 0$, la preparación *salta* el modo lento y relaja a la tasa rápida $\Rightarrow$ adelanta a otra que sí lo tiene [5, 6].
- La **no-normalidad** de $\mathcal{L}$ ($\mathcal{L}\mathcal{L}^\dagger \neq \mathcal{L}^\dagger\mathcal{L}$): los modos no son ortogonales $\Rightarrow$ permite el adelantamiento.

Predicción del cruce: del espectro, *sin evolucionar*

Juan

Definimos dos preparaciones (estados producto puros), ambas relajando al *mismo* estacionario verdadero ρ_{ss} :

- $\rho_A = |+\rangle^{\otimes N}$, con $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ — **parte LEJOS**.
- $\rho_B = |0\rangle^{\otimes N}$ — **parte CERCA**.

Con el **espectro** (de T1: $\lambda_2, \hat{l}_2$) y los **solapamientos** $a_k = \text{Tr}(\hat{l}_k^\dagger \rho_0)$ —*solo una traza, sin integrar*— ya conocemos el destino de cada estado:

estado	D_0 (arranque)	$ a_2 $ (peso del modo lento $\lambda_2 = -0.70$)
$ +\rangle^{\otimes N}$ (lejos)	0.99	0.13
$ 0\rangle^{\otimes N}$ (cerca)	0.84	0.50

$|+\rangle$ **arranca más alto** ($\times 1.18$) pero **pesa menos en el modo lento** (asintótico $\times 0.27$) $\Rightarrow$ **cruce garantizado**, con $t_{\text{pred}}^* \approx 0.68$, *antes de resolver la dinámica*. Las tareas T2/T3 solo lo *confirman*.

2 · T1 — Modos lentos del Liouvilliano

Juan

Objetivo. Obtener $\lambda_2, \lambda_3, \dots$ y los solapamientos a_k *sin diagonalizar* la matriz $4^N \times 4^N$ (inviabile en denso).

Método: Arnoldi sobre el propagador $P(\tau) = e^{\tau\mathcal{L}}$ [16, 12, 2].

- Los modos *lentos* de $\mathcal{L}$ (interiores, cerca de 0) son los *dominantes* de P ($\mu_k = e^{\tau\lambda_k}$) $\Rightarrow$ Arnoldi los extrae **primero**: “*faster than the clock*”.
- La acción de P es **matrix-free**: se integra $\dot{V} = \mathcal{L}[V]$ con RK4, sin formar $\mathcal{L}$.

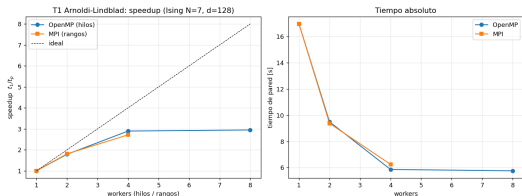
Serial

```
// producto C = A*B
for (int i=0;i<d;++i)
  for (int k=0;k<d;++k){
    cd a=A[i*d+k];
    for (int j=0;j<d;++j)
      C[i*d+j]+=a*B[k*d+j];
  }
```

Paralelo (datos)

```
#pragma omp parallel for          // intranodo
for (int i=rd.lo;i<rd.hi;++i)    // bloque del rango
  for (int k=0;k<d;++k){
    cd a=A[i*d+k];
    for (int j=0;j<d;++j)
      C[i*d+j]+=a*B[k*d+j];
  }
MPI_Allgatherv(...);           // reune C (internodo)
```

Dos niveles: cada **rango MPI** calcula un *bloque de filas*, **OpenMP** lo reparte entre hilos, y **MPI_Allgatherv** reconstruye la matriz. El SpMV de Arnoldi llama a este núcleo.



Escalado fuerte (Ising $N = 7$, $d = 128$).

Computacional.

- OpenMP $2.9\times$ / MPI $2.7\times$ (4 workers); *plateau* en los núcleos físicos.
- SpMV *memory-bound* y $\mathcal{L}$ **acoplado** $\Rightarrow$ comunicación Allgatherv: es el caso *menos* paralelizable.

Validación / física.

- Error en λ lentos: 10^{-13} – 10^{-5} vs denso.
- *Faster-than-the-clock* hasta $8.8\times$ en SpMV.
- Da el λ_2 y el a_2 que **determinan el cruce** de Mpemba.

3 · Evolución temporal (a): RK4 directo

Sebastián

Framework. Integrar $\dot{\rho} = \mathcal{L}[\rho]$ desde una preparación ρ_0 y trazar la distancia $D_{HS}(\rho_t \parallel \rho_{ss})$ al estacionario.

Discretización: Runge–Kutta 4 (RK4). Con paso Δt ,

$$\rho_{n+1} = \rho_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4), \quad k_i = \mathcal{L}[\cdot],$$

y la acción **matrix-free** $\mathcal{L}[V] = -i(HV - VH) + \sum_{\mu} (L_{\mu} V L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, V\})$.

- Orden global $\mathcal{O}(\Delta t^4)$; coste por paso $\mathcal{O}(Nd^3)$ (domina el **producto de matrices** densas).

- **Patrón:** miles de pasos con **dependencia temporal estricta** (ρ_{n+1} necesita ρ_n) $\Rightarrow$ *no* se paraleliza entre pasos.
- El paralelismo disponible está *dentro* del paso, en el `matmul`: datos regulares de tamaño moderado que caben en un nodo.
- $\Rightarrow$ paradigma natural **OpenMP** (memoria compartida, sin halo). **MPI** sería contraproducente (comunicación en cada uno de los miles de pasos); **CUDA** ayuda pero requiere GPU (lo vemos en la sección 5).
- **Correctitud:** serial y paralelo dan el *mismo* estado final a precisión de máquina (el observable D_{HS} coincide).

Serial

```
Mat matmul(A,B,d){
  Mat C(d*d);
  for(int i=0;i<d;++i)
    for(int k=0;k<d;++k){
      cd a=A[i*d+k];
      for(int j=0;j<d;++j)
        C[i*d+j]+=a*B[k*d+j];}
  return C;}

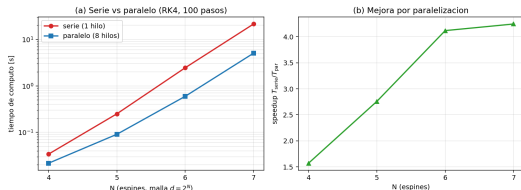
```

Paralelo

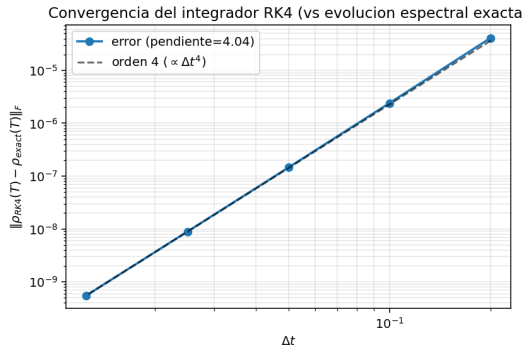
```
Mat matmul(A,B,d){
  Mat C(d*d);
  #pragma omp parallel for schedule(static)
  for(int i=0;i<d;++i)
    for(int k=0;k<d;++k){
      cd a=A[i*d+k];
      for(int j=0;j<d;++j)
        C[i*d+j]+=a*B[k*d+j];}
  return C;}

```

Una `#pragma omp parallel for` reparte las filas i entre hilos. El *mismo* binario, con distinto `OMP_NUM_THREADS`, da serie vs paralelo.



Serie vs paralelo vs N : speedup $\rightarrow 4\times$ (sublineal).

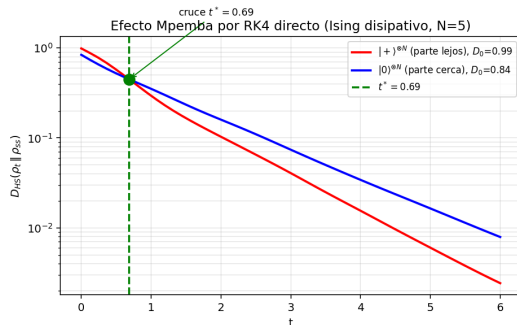


Convergencia orden 4 (≈ 4.04) vs solución exacta.

Speedup OpenMP *bandwidth-bound*; el integrador queda validado a orden 4 contra la evolución espectral exacta.

T2a — Resultado físico: el cruce de Mpemba

Sebastián



Dos preparaciones del mismo modelo: $|+\rangle^{\otimes N}$ (parte lejos) y $|0\rangle^{\otimes N}$ (parte cerca), distancia al **estacionario verdadero** ρ_{SS} .

La que parte lejos (roja) **adelanta** a la cercana (azul) en $t^* \approx 0.69$: ahí está el efecto Mpemba, reproducido por RK4.

4 · Evolución temporal (b): Krylov

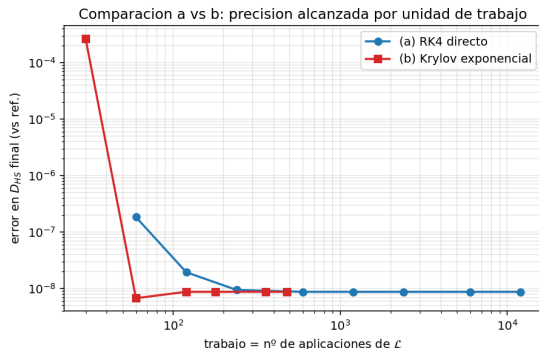
Sebastián

Idea. En vez de integrar paso a paso, aplicar la **acción del exponencial** $\rho(t+\tau) = e^{\tau\mathcal{L}}[\rho]$ (solución *exacta* en un paso τ).

- $e^{\tau\mathcal{L}}$ es enorme $\Rightarrow$ se proyecta en un subespacio de **Krylov** $\mathcal{K}_m$ (Arnoldi, $m \sim 30$) y se exponencia la matriz pequeña $m \times m$ [20, 1].
- *Mismo hotspot* que RK4 (el `matmul` de $\mathcal{L}[\cdot]$) $\Rightarrow$ **misma paralelización OpenMP** y escalado equivalente.
- Ventajas: pasos τ grandes, **incondicionalmente estable**, ideal para llegar al estacionario.

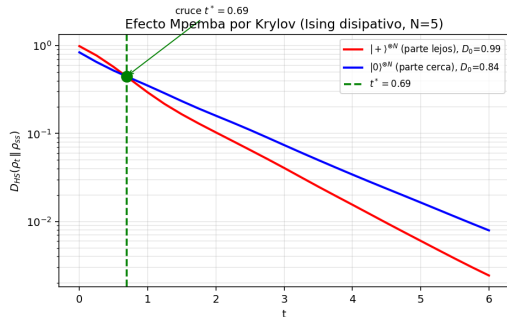
T2b — El resultado central: precisión por unidad de trabajo

Sebastián



A igual **trabajo** (n° de aplicaciones de $\mathcal{L}$, independiente de la máquina), **Krylov es $\sim 25\times$ más preciso** que RK4.

Convergencia *super-algebraica* en m frente a la algebraica ($\propto \Delta t^4$) de RK4. RK4 sigue siendo mejor para trayectorias densas o $\mathcal{L}$ dependiente del tiempo.



Mismo cruce, $t^* \approx 0.69$, idéntico a RK4 (acuerdo a 10^{-10} en D_{HS} final).

Mensaje: el cruce es **física**, no un artefacto del integrador; y Krylov lo obtiene con pasos de tiempo grandes.

5 · Muchos cuerpos: trayectorias (MPI) + GPU

Sebastián

Monte Carlo wavefunction [7, 10]: propagar M estados *puros* $|\psi\rangle \in \mathbb{C}^d$ (memoria d , no d^2) con H_{eff} y saltos, y promediar $\rho = \mathbb{E}[|\psi\rangle\langle\psi|]$. **Embarrassingly parallel**.

Serial

```
for(int tr=0;tr<M;++tr){
  psi=ket0;
  for(paso) RK4_Heff+salto(psi);
  acc += |psi><psi|;
}
rho = acc / M;
```

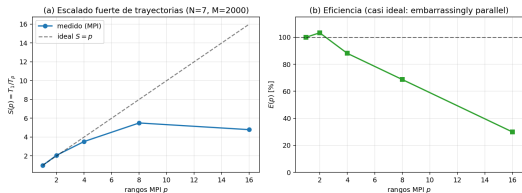
Paralelo (MPI)

```
int my_M = M/size + ...;    // reparto
for(int tr=0;tr<my_M;++tr){
  ... acc_local += |psi><psi|;
}
MPI_Reduce(acc_local,acc,MPI_SUM,0);
if(rank==0) rho = acc / M;  // 1 sola colectiva
```

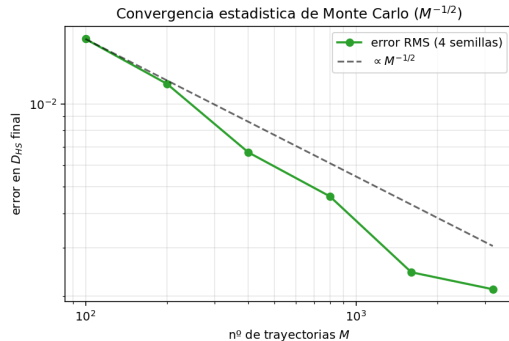
Cada rango integra su lote *sin comunicación*; una única **MPI_Reduce** promedia. Sin halo ni dependencias $\Rightarrow$ escalado casi ideal y multinodo.

T3 — Escalado y convergencia estadística

Sebastián



Escalado fuerte MPI: *casi ideal* ($S \approx 5.5$).

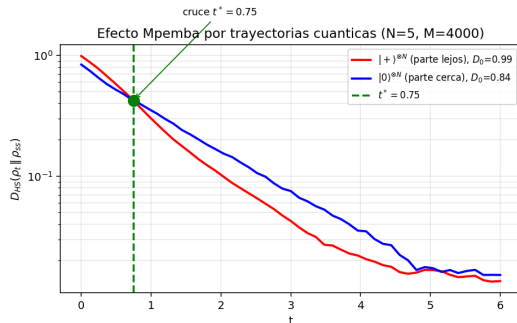


Convergencia estadística $\propto M^{-1/2}$.

Contraste con el OpenMP sublineal de T2: **el patrón de cómputo dicta el paradigma**. Precisión: una cifra más $\Rightarrow 100\times$ trayectorias (pero hay paralelismo de sobra).

T3 — Resultado físico: el cruce de Mpemba

Sebastián

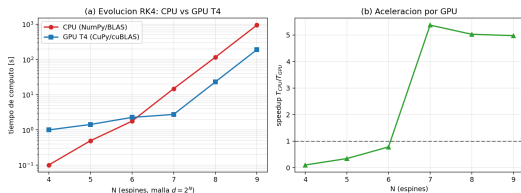


También el método *estocástico* captura el cruce: la curva que parte lejos **adelanta** a la cercana en $t^* \approx 0.75$.

El ruido del suelo a tiempos largos es el error estadístico $M^{-1/2}$ del Monte Carlo.

T3 — Aceleración en GPU (CUDA / T4)

Sebastián



Evolución RK4: CPU vs GPU T4 (cuBLAS).

Tercer escalón: serie $\rightarrow$ OpenMP $\rightarrow$ **CUDA T4**. El matmul denso $\rightarrow$ cuBLAS zgemm.

- GPU pierde a d pequeño (kernels) y **gana** $\sim 5\times$ a $N \geq 7$ ($N=9$: 943 s $\rightarrow$ 190 s).
- GEMM hasta 250 GFLOP/s ($d=2048$).
- Validación triple GPU = CPU = C++ ($D_{HS} \rightarrow 0$).

Notebook ejecutado + resultados_t2_cuda.zip en T2/cuda/.

6 · Muchos cuerpos: redes tensoriales

Juan

Redes tensoriales [22, 19]. La matriz densidad como un *superket* $|\rho\rangle\rangle$ — un MPS de dimensión física 4 por sitio:

- **TEBD**: se trotteriza $e^{dt \mathcal{L}}$ en **compuertas locales** de 2 sitios y se **trunca** el enlace a χ por SVD.
- Coste **polinómico en χ** (no exponencial en N) $\Rightarrow$ abre tamaños imposibles en denso.
- **Paralelización: hilos**. En una capa, los bonds *pares* (o *impares*) son disjuntos; sus SVD ($\mathcal{O}(\chi^3)$) son independientes y numpy **libera el GIL**.

Serial

```
# capa de Trotter
for k in bonds:      # disjuntos
    mps.apply_gate(
        k, gate, chi) # SVD  $O(\chi^3)$ 
```

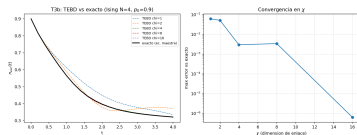
Paralelo (ThreadPool)

```
executor.map(
    lambda k: mps.apply_gate(k, gate, chi),
    bonds)      # SVD libera el GIL
# capas par / impar -> bonds independientes
```

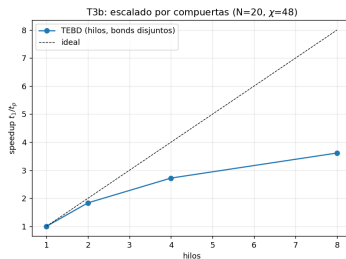
Paralelismo **estructurado**: intermedio entre el SpMV acoplado (T1) y las trayectorias independientes (T3). Se mapea a MPI repartiendo segmentos de la cadena.

T3b — Validación, escalado y alcance

Juan

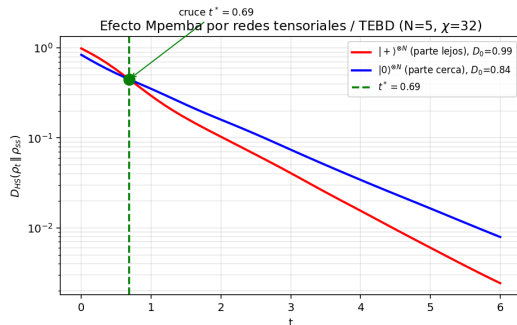


Error $\rightarrow 6 \times 10^{-7}$ al subir χ .



Hilos: $3.6 \times (8)$.

- Validado vs exacto: error controlado por χ .
- Escala $\sim 3.6 \times$ (estructurado).
- $N = 32$ ($4^{32} \approx 10^{19}$), imposible en denso, en ~ 3.6 s.



TEBD también reproduce el efecto: $|+\rangle^{\otimes N}$ adelanta a $|0\rangle^{\otimes N}$ en $t^* \approx 0.69$, idéntico a RK4/Krylov (a χ grande TEBD es exacto).

Los cuatro métodos de evolución (RK4, Krylov, trayectorias, redes tensoriales) dan el mismo t^* .

7 · Conclusiones generales

Juan

- **El patrón de cómputo dicta el paradigma.** Lo vimos como un espectro de escalabilidad:
 - **T1** SpMV *acoplado* (OpenMP/MPI $\sim 2.9\times$, comunicación).
 - **T3b** redes tensoriales *estructurado* (hilos $\sim 3.6\times$).
 - **T3** trayectorias *embarrassingly parallel* (MPI $\sim 5.6\times$, casi ideal, multinodo).
 - **T2** evolución densa $\rightarrow$ **GPU** (cuBLAS $\sim 5\times$).
- Cada paradigma (OpenMP, MPI, hilos, CUDA) elegido por la *estructura del algoritmo*, no por moda. *Acoplado* $<$ *estructurado* $<$ *independiente*.

- El efecto Mpemba cuántico es **geometría espectral**: no-normalidad de $\mathcal{L}$ + criterio $a_2 = 0$ [5].
- **Cruce reproducido por los CUATRO métodos** de evolución (RK4, Krylov, trayectorias, redes tensoriales), todos con $t^* \approx 0.69\text{--}0.75$ — **validación cruzada** total.
- Escalable a muchos cuerpos por dos rutas complementarias (trayectorias y redes tensoriales) que llegan a tamaños imposibles en denso.

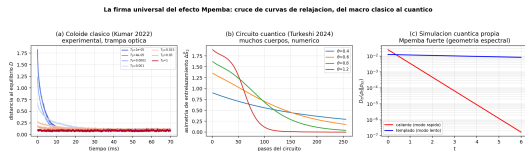
8 · Relevancia en el efecto macro/meso

Sebastián

La firma es universal. El mismo mecanismo — **geometría espectral de la relajación** — explica el cruce en cascadas clásicas, coloides y sistemas cuánticos [14, 11, 21].

Lo que devuelve el enfoque cuántico:

- Un *criterio operacional* ($a_2 = 0$) y herramientas HPC para *predecir* y *diseñar* el efecto.
- Aplicaciones: **termometría cuántica** y enfriamiento acelerado [15, 9].



La firma universal del cruce: del macro al cuántico.

Referencias I



A. H. Al-Mohy and N. J. Higham.

Computing the action of the matrix exponential, with an application to exponential integrators.
SIAM Journal on Scientific Computing, 33(2):488–511, 2011.



S. Balay et al.

PETSc/TAO users manual.

Technical Report ANL-21/39 Revision 3.20, Argonne National Laboratory, 2023.



J. Bechhoefer, A. Kumar, and R. Chetrite.

A fresh understanding of the Mpemba effect.
Nature Reviews Physics, 3:534–535, 2021.



H.-P. Breuer and F. Petruccione.

The Theory of Open Quantum Systems.
Oxford University Press, 2002.



F. Carollo, A. Lasanta, and I. Lesanovsky.

Exponentially accelerated approach to stationarity in Markovian open quantum systems through the Mpemba effect.
Physical Review Letters, 127:060401, 2021.



P. Chattopadhyay, J. F. G. Santos, and A. Misra.

Anomaly to resource: the Mpemba effect in quantum thermometry.
arXiv preprint arXiv:2601.05046, 2026.

Referencias II

- 
- A. J. Daley.
Quantum trajectories and open many-body quantum systems.
Advances in Physics, 63(2):77–149, 2014.
- 
- V. Gorini, A. Kossakowski, and E. C. G. Sudarshan.
Completely positive dynamical semigroups of N-level systems.
Journal of Mathematical Physics, 17(5):821–825, 1976.
- 
- F. Ivander, N. Anto-Sztrikacs, and D. Segal.
Hyperacceleration of quantum thermalization dynamics by bypassing long-lived coherences.
Physical Review E, 108:014130, 2023.
- 
- J. R. Johansson, P. D. Nation, and F. Nori.
QuTiP 2: a Python framework for the dynamics of open quantum systems.
Computer Physics Communications, 184(4):1234–1240, 2013.
- 
- A. Kumar, R. Chétrite, and J. Bechhoefer.
Anomalous heating in a colloidal system.
Proceedings of the National Academy of Sciences, 119(5):e2118484119, 2022.
- 
- R. B. Lehoucq, D. C. Sorensen, and C. Yang.
ARPACK users' guide: Solution of large-scale eigenvalue problems with implicitly restarted Arnoldi methods.
SIAM, 1998.

Referencias III



G. Lindblad.

On the generators of quantum dynamical semigroups.

Communications in Mathematical Physics, 48(2):119–130, 1976.



Z. Lu and O. Raz.

Nonequilibrium thermodynamics of the Markovian Mpemba effect and its inverse.

Proceedings of the National Academy of Sciences, 114(20):5083–5088, 2017.



S. K. Manikandan.

Equidistant quenches in few-level quantum systems.

Physical Review Research, 3:043108, 2021.



F. Minganti, A. Biella, N. Bartolo, and C. Ciuti.

Spectral theory of Liouvillians for dissipative phase transitions.

Physical Review A, 98:042118, 2018.



E. B. Mpemba and D. G. Osborne.

Cool?

Physics Education, 4(3):172–175, 1969.



A. Nava and R. Egger.

Mpemba effects in open nonequilibrium quantum systems.

Physical Review Letters, 133:136302, 2024.

Referencias IV



R. Orús.

A practical introduction to tensor networks: matrix product states and projected entangled pair states.
Annals of Physics, 349:117–158, 2014.



R. B. Sidje.

Expokit: a software package for computing matrix exponentials.
ACM Transactions on Mathematical Software, 24(1):130–156, 1998.



G. Teza, J. Bechhoefer, A. Lasanta, O. Raz, and M. Vucelja.

Speedups in nonequilibrium thermal relaxation: Mpemba and related effects.
Physics Reports, 1164:1–97, 2026.



H. Weimer, A. Kshetrimayum, and R. Orús.

Simulation methods for open quantum many-body systems.
Reviews of Modern Physics, 93:015008, 2021.

¡Gracias!

Juan | Sebastián — Efecto Mpemba cuántico en HPC